

Relatório de Criptografia 1

- ROT-X

Sabe-se que as três primeiras letras da mensagem original são “CTF”. Sendo a criptografia utilizada a de ROT, podemos descobrir qual foi o valor ROT-X utilizado para codificar a mensagem apenas verificando quantos “shifts” ocorreram para que as três primeiras letras da mensagem original “CTF” virassem “RIU”. Analisando apenas a conversão de C para R, percebemos que foram 15 shifts para a direita, dessa forma, a criptografia utilizada foi a de ROT-15. Para decodificar a mensagem, basta realizar o ROT do complementar desse valor em relação ao alfabeto, ou seja, ROT-(26-X), que será de ROT-11. Utilizei o site rot13.com para fazer a decodificação, segue a imagem abaixo.

rot13.com

[About ROT13](#)

RIU-QG{P_B4s1r@d_s4_B4chp0_Qan}



ROT11 ▾



CTF-BR{A_M41d1c@o_d4_M4nsa0_B1y}

- The rest is confetti

A cifra utilizada nesse chall foi a Cifra de XOR, como foi possível verificar pelo uso das notações “keyA”, “keyB” e “keyC”. Para decodificação de ciphertxts que utilizaram Cifra de XOR em sua codificação, basta aplicar XOR com as mesmas chaves na mensagem codificada, em seguida realizar a conversão de long to bytes (oposto da bytes to long que foi usado na codificação). Segue abaixo a imagem do código, a alteração está na linha 9.

```
confetti.py X
confetti.py > ...
1 from Crypto.Util.number import long_to_bytes
2 FLAG = b"XXXX"
3 ciphertext = 34797035105098139296910771775520259408031966125812008802071645385032929448104379556598639
4
5 keyA = 80702602140020670882597065913281334107440667458180810708153958923730944057275073737969808
6 keyB = 57958375141770915463809884541547382649493525070950388713927258233583028115302863328704290
7 keyC = 7322298043641924711193667891633174162550682088244909427377837843656106030975992433793184
8
9 FLAG = long_to_bytes(ciphertext ^ keyA ^ keyB ^ keyC)
10 print(FLAG)
```

- Stuart Little

A cifra utilizada nesse chall foi o RSA, com módulo $n = 2q$, como descrito no problema. Sendo os valores de n e e públicos, é possível utilizar esses valores para deduzir as chaves privadas q , $\Phi(n)$ e d . Para o valor de q , basta utilizarmos o valor de n na fórmula $n = 2q$ que descobrimos o valor de q (a divisão no código é inteira para evitar problema de ponto flutuante). Para descobrirmos o valor de $\Phi(n)$, basta utilizarmos a Função de Euler aplicada para $n = pq$, na qual encontramos a expressão $\Phi(n) = (p - 1)(q - 1)$ e que pode ser ainda mais simplificada sabendo que $p = 2$, assim ficando $\Phi(n) = q - 1$. Por fim, o valor de d é o inverso modular de $e \bmod \Phi(n)$, assim, podemos utilizar a função “pow” do Python para calcular essa exponenciação modular, no qual colocamos como parâmetros a base, o expoente e o módulo, respectivamente e , -1 e $\Phi(n)$. Segue abaixo a imagem do código.

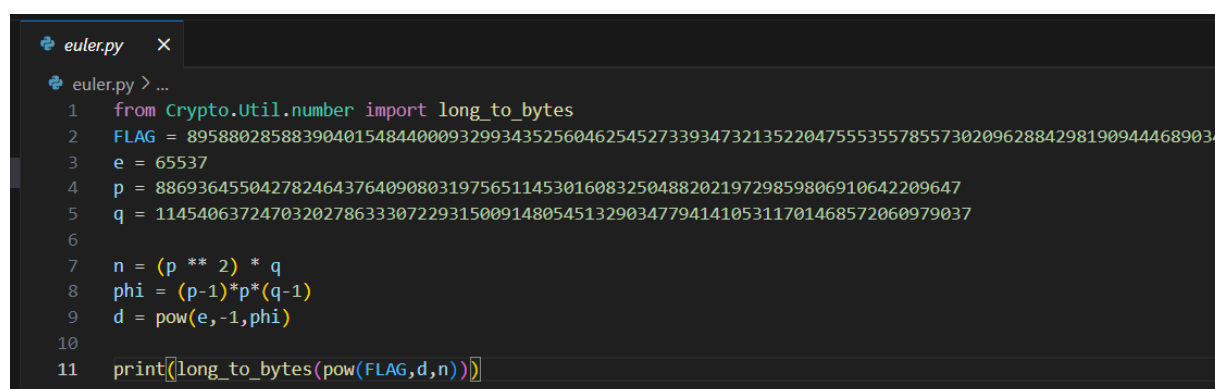
```
stuart.py X
stuart.py > ...
1 from Crypto.Util.number import long_to_bytes
2 FLAG = 31629006585085327409812507270875453196064125651594425718867181172787427788449 # mensagem cifrada = c
3 e = 65537 # expoente publico
4
5 p = 2
6 n = 174273493446054651350782275296370226885594728696881571385688057497125122365534
7 q = n//2 # q = n/2 -> n = p.q, logo q = n/p = n/2
8 phi = q-1 # phi = (p-1)(q-1), p = 2
9 d = pow(e,-1,phi)
10 print(long_to_bytes(pow(FLAG, d, n)))
```

- Euler

Para esse chall, bastava complementar o código com os valores de $\Phi(n)$ e d . Para o valor de d , a fórmula e implementação é a mesma utilizada e explicada no chall anterior, na qual d equivale ao inverso modular de $e \bmod \Phi(n)$, e assim pode ser escrito em código a partir do “pow” do Python. Já para $\Phi(n)$, é apenas uma aplicação da função totiente de Euler sobre a equação de n fornecida, seguindo a seguinte matemática:

$$\Phi(n) = (p - 1) \cdot p^{2-1} \cdot (q - 1) \cdot q^{1-1} = (p - 1) \cdot p \cdot (q - 1)$$

Segue abaixo a imagem do código feito:



```
euler.py
1 from Crypto.Util.number import long_to_bytes
2 FLAG = 895880285883904015484400093299343525604625452733934732135220475553557855730209628842981909444689034
3 e = 65537
4 p = 88693645504278246437640908031975651145301608325048820219729859806910642209647
5 q = 114540637247032027863330722931500914805451329034779414105311701468572060979037
6
7 n = (p ** 2) * q
8 phi = (p-1)*p*(q-1)
9 d = pow(e,-1,phi)
10
11 print(long_to_bytes(pow(FLAG,d,n)))
```

- Teorema Chinês do Resto

Para esse chall, precisamos complementar o código fornecido com os valores das variáveis N_i , x_i e X . Isso acaba se resumindo a aplicação das fórmulas relacionadas ao Teorema Chinês do Resto, que encontrei ao ler o PDF disponibilizado sobre esse tema no GET de Criptografia. Segue abaixo as fórmulas para cada variável:

Demonstração: Seja $N = n_1 \dots n_k$ e $N_i = \frac{N}{n_i}$. Note que $\text{mdc}(n_i, N_i) = 1$, então pelo **Lema 4.4** existem $x_i, y_i \in \mathbb{Z}$ tais que $N_i \cdot x_i + n_i \cdot y_i = 1$. Segue disso, que $N_i \cdot x_i \equiv 1 \pmod{n_i} \implies (N_i \cdot a_i)x_i \equiv a_i \pmod{n_i}$. Perceba que se $i \neq j$, então $N_i \equiv 0 \pmod{n_j}$, visto que N_i é o produto de todos n_k exceto n_i , então se $j \neq i$ temos que $N_i = (y) \cdot n_j$ é um múltiplo de n_j e por definição é congruente a zero. Denote $X = N_1 \cdot a_1 \cdot x_1 + \dots + N_k \cdot a_k \cdot x_k$ e tome $j \in \{1, \dots, k\}$, obtemos $X \equiv N_j \cdot a_j \cdot x_j \pmod{n_j}$ pela observação anterior. Como $(N_j \cdot a_j) \cdot x_j \equiv a_j \pmod{n_j}$, concluímos que $X \equiv a_j \pmod{n_j}$.

Figura 1 - Fórmula para N_i , na demonstração da página 7

Na demonstração do **Teorema 6.1** vimos que $N_i \cdot x_i \equiv 1 \pmod{n_i}$, faremos isso então. Sejam $n_1 = 9$, $n_2 = 10$, $N_1 = \frac{9 \cdot 10}{9} = 10$ e $N_2 = \frac{10 \cdot 9}{10} = 9$, temos $N_1 \cdot x_1 \equiv 1 \pmod{n_1}$ e $N_2 \cdot x_2 \equiv 1 \pmod{n_2}$, isto é, $10 \cdot x_1 \equiv 1 \pmod{9}$ e $9 \cdot x_2 \equiv 1 \pmod{10}$.

Facilmente encontramos $x_1 \equiv 1 \pmod{n_1}$, pois $10 \equiv 1 \pmod{9}$ e $x_2 \equiv 9 \pmod{10}$, porque $81 \equiv 1 \pmod{9}$. Por fim, precisamos calcular $X \equiv \sum N_i x_i a_i \pmod{N}$. Assim, $X \equiv 10 \cdot 1 \cdot 8 + 9 \cdot 9 \cdot 3 \equiv 323 \pmod{N}$. Como $N = n_1 \times n_2 = 10 \cdot 9 = 90$, segue que $X \equiv 53 \pmod{90}$.

Figura 2 - Fórmulas para x_i e X , no Exemplo Prático 1 da página 7

Em relação à implementação em código, N_i é apenas uma divisão como prevista na fórmula, sendo ela inteiro para evitar problemas com ponto flutuante. Por outro lado, o valor de x_i é igual ao inverso modular de $N_i \bmod n_i$, uma forma de apresentar o valor de x_i a partir da fórmula mostrada anteriormente, assim, pode ser calculado no Python a partir do “pow” no qual N_i é a base, -1 é o expoente para calcular o inverso modular e n_i é o módulo. Por fim, X é o somatório dos produtos das variáveis N_i , x_i e a_i , dessa forma, basta colocar o produto dessas três variáveis com o operador $+=$ que a variável computará o somatório desses produtos.

Segue abaixo o código:

```
CRT.py > ...
1  import math
2  from Crypto.Util.number import long_to_bytes
3  # Altere somente as linhas com ??
4  def CRT(sistema):
5      N = 1
6      for ai, ni in sistema:
7          N *= ni
8
9      X = 0
10     for ai, ni in sistema:
11         Ni = N//ni
12         xi = pow(Ni, -1, ni) # xi * Ni ≡ 1 (mod ni)
13         X += Ni * xi * ai
14
15     return pow(X,1,N)
16
17  e = 7
18  n1 = 12836199686973860671317829319572068536675366202551
19  n2 = 9207164809729988062334325079856956174096146645421
20  n3 = 13622149579754221729460065730654953951867472393417
21
22  a1 = 3813911391918830343786624637400279512933046231999
23  a2 = 2068264257340920848271568225822036140145353768448
24  a3 = 5020458554640173667230974368342025301857250930440
25
26  sistema = [(a1, n1),
27             (a2, n2),
28             (a3, n3)] # X ≡ a1 (mod n1), X ≡ a2 (mod n2), X ≡ a3 (mod n3)
29
30  X = math.ceil(CRT(sistema)**(1/e)) # X ≡ (msg**7)**(1/7) (mod N)
31
32  print(f"CTF-BR{{{long_to_bytes(X).decode()}}}")
```